

```

function getLangInstruction(lang) {
  if (lang === "hi") return "Reply in simple Hindi.";
  if (lang === "mr") return "Reply in simple Marathi.";
  return "Reply in English.";
}

function doGet(e) {
  const ss = SpreadsheetApp.getActive();
  const users = ss.getSheetByName("Users");
  const billing = ss.getSheetByName("Billing");

  const mode = e.parameter.mode;
  const uid = e.parameter.uid;

  // ----- START SESSION -----
  if (mode === "start") {
    billing.clearContents();
    billing.appendRow(["UID", "Item", "Price", "Quantity", "Total"]);

    // Find user email
    const data = users.getDataRange().getValues();
    let email = "";

    for (let i = 1; i < data.length; i++) {
      if (String(data[i][0]).trim() === String(uid).trim()) {
        email = data[i][3]; // Column D = Email
        break;
      }
    }

    // Send scan link
    if (email) {
      const scanLink = "https://jolly-sunshine-587951.netlify.app/?uid="
+ uid;

      const subject = "Smart Cart â€” Start Scanning Items";
      const body =
        "Hi!\n\n" +
        "Your shopping session has started.\n" +
        "Click the link below to scan items using your phone:\n\n" +
        scanLink +
        "\n\nHappy Shopping!";

      MailApp.sendEmail(email, subject, body);
      return ContentService.createTextOutput("Session Started + Email
Sent");
    }

    return ContentService.createTextOutput("Session Started (no email
found)");
  }

  // ----- ADD ITEM -----
  if (mode === "add") {
    const item = e.parameter.item;
    const price = Number(e.parameter.price);
    const qty = Number(e.parameter.qty || 1);

```

```

        billing.appendRow([uid, item, price, qty, price * qty]);
        return ContentService.createTextOutput("Item Added");
    }

    // ----- END SESSION -----
    if (mode === "end") {
        const lastRow = billing.getLastRow();
        let total = 0;

        if (lastRow > 1) {
            // Sum TOTAL column (E = 5)
            const totals = billing.getRange(2, 5, lastRow - 1, 1).getValues();
            total = totals.reduce((sum, r) => sum + Number(r[0]), 0);
        }

        // Deduct from wallet + get email
        const usersData = users.getDataRange().getValues();
        let email = "";

        for (let i = 1; i < usersData.length; i++) {
            if (String(usersData[i][0]).trim() === String(uid).trim()) {
                users.getRange(i + 1, 3).setValue(usersData[i][2] - total); //
                email = usersData[i][3]; // Email in col D
                break;
            }
        }

        // Send bill email
        if (email) {
            const subject = "Smart Cart â€” Your Bill";
            const body =
                "Thank you for shopping!\n\n" +
                "Your total bill is: â‚¬" + total + "\n\n" +
                "Amount has been deducted from your wallet.\n\n" +
                "Visit again!";

            MailApp.sendEmail(email, subject, body);
        }

        return ContentService.createTextOutput("Total Bill: " + total);
    }

    // ----- REMOVE ITEM -----
    if (mode === "remove") {
        const item = e.parameter.item;
        const uid = e.parameter.uid;

        const sheet = SpreadsheetApp.getActive().getSheetByName("Billing");
        const data = sheet.getDataRange().getValues();

        // start from bottom so row index stays safe
        for (let i = data.length - 1; i > 0; i--) {
            if (
                String(data[i][0]).trim() === String(uid).trim() &&
                String(data[i][1]).trim() === String(item).trim()
            ) {
                sheet.deleteRow(i + 1);
                break; // remove only one entry
            }
        }
    }

```

```

    }
}

return ContentService.createTextOutput("Item Removed");
}

// ----- GET PRODUCT DETAILS -----
if (mode === "get_product") {
    const code = e.parameter.code;
    const sheet = SpreadsheetApp.getActive().getSheetByName("Products");

    const data = sheet.getDataRange().getValues();
    for (let i = 1; i < data.length; i++) {
        if (String(data[i][0]).trim() === String(code).trim()) {
            return ContentService.createTextOutput(
                JSON.stringify({
                    name: data[i][1],
                    price: data[i][2]
                })
            );
        }
    }

    // fallback if not found
    return ContentService.createTextOutput(
        JSON.stringify({ name: "Unknown Item", price: 0 })
    );
}

// ----- GET PRODUCT LOCATION -----
if (mode === "get_location") {
    const code = e.parameter.code;
    const sheet = SpreadsheetApp.getActive().getSheetByName("Products");

    const data = sheet.getDataRange().getValues();
    for (let i = 1; i < data.length; i++) {
        if (String(data[i][0]) === String(code)) {
            const locationText =
                `${data[i][1]} is located at ${data[i][3]}, ${data[i][4]},`
            + `${data[i][5]}.`;
            return ContentService.createTextOutput(locationText);
        }
    }

    return ContentService.createTextOutput("Sorry, location not found for this product.");
}

// ----- AI RECIPE ASSISTANT -----
if (mode === "ai_recipe") {
    const billing = SpreadsheetApp.getActive().getSheetByName("Billing");
    const lastRow = billing.getLastRow();

    if (lastRow < 2) {
        return ContentService.createTextOutput("Your cart is empty.");
    }
}

```

```

// Read items from Billing sheet
const data = billing.getRange(2, 2, lastRow - 1, 4).getValues();
const items = data.map(r => `${r[0]} x${r[2]}`).join(", ");

const lang = e.parameter.lang || "en";
const langInstruction = getLangInstruction(lang);

const prompt =
  langInstruction + " " +
  "I have these items in my grocery cart: " + items +
  ". Suggest 2-3 easy Indian recipes I can cook. " +
  "Give short steps.";

try {
  const aiText = callGroq(prompt);
  return ContentService.createTextOutput(aiText);
} catch (err) {
  return ContentService.createTextOutput("AI Error: " + err.message);
}
}

// ----- AI NUTRITION ADVISOR -----
if (mode === "ai_nutrition") {
  const billing = SpreadsheetApp.getActive().getSheetByName("Billing");
  const lastRow = billing.getLastRow();

  if (lastRow < 2) {
    return ContentService.createTextOutput("Your cart is empty.");
  }

  const data = billing.getRange(2, 2, lastRow - 1, 4).getValues();
  const items = data.map(r => `${r[0]} x${r[2]}`).join(", ");

  const lang = e.parameter.lang || "en";
  const langInstruction = getLangInstruction(lang);

  const prompt =
    langInstruction + " " +
    "Analyze the nutrition of these grocery items: " + items + ". " +
    "Tell me if the cart is healthy or not. " +
    "Warn about high sugar, junk food, or excess oil. " +
    "Suggest healthier Indian alternatives if needed. " +
    "Keep it simple and friendly.";

  try {
    const aiText = callGroq(prompt);
    return ContentService.createTextOutput(aiText);
  } catch (err) {
    return ContentService.createTextOutput("AI Error: " + err.message);
  }
}

// ----- AI SMART BUDGET -----
if (mode === "ai_budget") {

```

```

const billing = SpreadsheetApp.getActive().getSheetByName("Billing");
const lastRow = billing.getLastRow();

if (lastRow < 2) {
  return ContentService.createTextOutput("Your cart is empty.");
}

const data = billing.getRange(2, 2, lastRow - 1, 4).getValues();
const items = data.map(r => `${r[0]} x${r[2]} (â,¹${r[3]})`).join(",
");

const lang = e.parameter.lang || "en";
const langInstruction = getLangInstruction(lang);

const prompt =
  langInstruction + " " +
  "These items are in a grocery cart: " + items + ". " +
  "Suggest cheaper or smarter buying alternatives in an Indian store. "
+
  "Recommend local brands, loose items, or quantity changes. " +
  "Keep it short and practical.";

try {
  const aiText = callGroq(prompt);
  return ContentService.createTextOutput(aiText);
} catch (err) {
  return ContentService.createTextOutput("AI Error: " + err.message);
}
}

// ----- AI BILL EXPLANATION -----
if (mode === "ai_bill") {
  const billing = SpreadsheetApp.getActive().getSheetByName("Billing");
  const lastRow = billing.getLastRow();

  if (lastRow < 2) {
    return ContentService.createTextOutput("Your cart is empty.");
  }

  const data = billing.getRange(2, 2, lastRow - 1, 4).getValues();
  let total = 0;
  const items = data.map(r => {
    total += Number(r[3]);
    return `${r[0]} x${r[2]} = â,¹${r[3]}`;
  }).join(", ");

  const lang = e.parameter.lang || "en";
  const langInstruction = getLangInstruction(lang);

  const prompt =
    langInstruction + " " +
    "A customerâ€™s grocery bill is â,¹" + total + ". " +
    "Items: " + items + ". " +
    "Explain simply why the bill is this amount. " +
    "Highlight costly items and quantity impact. " +
    "Suggest how to reduce the bill next time.";

  try {

```

```

        const aiText = callGroq(prompt);
        return ContentService.createTextOutput(aiText);
    } catch (err) {
        return ContentService.createTextOutput("AI Error: " + err.message);
    }
}

// ----- AI PRODUCT LOCATION -----
if (mode === "ai_location") {
    const itemName = e.parameter.item || "";
    const sheet = SpreadsheetApp.getActive().getSheetByName("Products");

    let location = "";
    const data = sheet.getDataRange().getValues();
    for (let i = 1; i < data.length; i++) {
        if (String(data[i][1]).toLowerCase() === itemName.toLowerCase()) {
            location = `${data[i][3]}, ${data[i][4]}, ${data[i][5]}`;
            break;
        }
    }

    if (!location) {
        return ContentService.createTextOutput("I couldnâ€™t find the location of that product.");
    }

    const lang = e.parameter.lang || "en";
    const langInstruction = getLangInstruction(lang);

    const prompt =
        langInstruction + " " +
        `A customer is asking where to find ${itemName} in the store. ` +
        `The product is located at ${location}. ` +
        `Answer politely and clearly.`;

    try {
        const aiText = callGroq(prompt);
        return ContentService.createTextOutput(aiText);
    } catch (err) {
        return ContentService.createTextOutput("AI Error: " + err.message);
    }
}

// ----- AI RECIPE LIST -----
if (mode === "ai_recipe_list") {
    const billing = SpreadsheetApp.getActive().getSheetByName("Billing");
    const lastRow = billing.getLastRow();

    if (lastRow < 2) {
        return ContentService.createTextOutput("Your cart is empty.");
    }

    const data = billing.getRange(2, 2, lastRow - 1, 4).getValues();
    const items = data.map(r => `${r[0]} x${r[2]}`).join(", ");

    const lang = e.parameter.lang || "en";
    const langInstruction = getLangInstruction(lang);

```

```

const prompt =
  langInstruction + " " +
  "I have these grocery items: " + items + ". " +
  "Suggest only 3 Indian recipe names I can cook. " +
  "Reply ONLY with recipe names in a list. No steps.";

return ContentService.createTextOutput(callGroq(prompt));
}
// ----- AI RECIPE STEPS -----
if (mode === "ai_recipe_steps") {
  const recipe = e.parameter.recipe;
  const lang = e.parameter.lang || "en";
  const langInstruction = getLangInstruction(lang);

  const prompt =
    langInstruction + " " +
    "Give simple step-by-step cooking instructions for " +
    recipe + ". Keep it short and easy.";

  return ContentService.createTextOutput(callGroq(prompt));
}

return ContentService.createTextOutput("Invalid Request");

}

function callGroq(prompt) {
  const apiKey = PropertiesService.getScriptProperties()
    .getProperty("GROQ_API_KEY");

  const url = "https://api.groq.com/openai/v1/chat/completions";

  const payload = {
    model: "llama-3.1-8b-instant",
    messages: [
      { role: "system", content: "You are a helpful Indian cooking assistant." },
      { role: "user", content: prompt }
    ],
    temperature: 0.6
  };

  const options = {
    method: "post",
    contentType: "application/json",
    headers: {
      Authorization: "Bearer " + apiKey
    },
    payload: JSON.stringify(payload),
    muteHttpExceptions: true
  };

  const res = UrlFetchApp.fetch(url, options);
  const text = res.getContentText();

```

```
Logger.log("Groq raw response: " + text);

const json = JSON.parse(text);

if (!json.choices) {
  return "AI service error: " + (json.error?.message || "Unknown
error");
}

return json.choices[0].message.content;
}
```